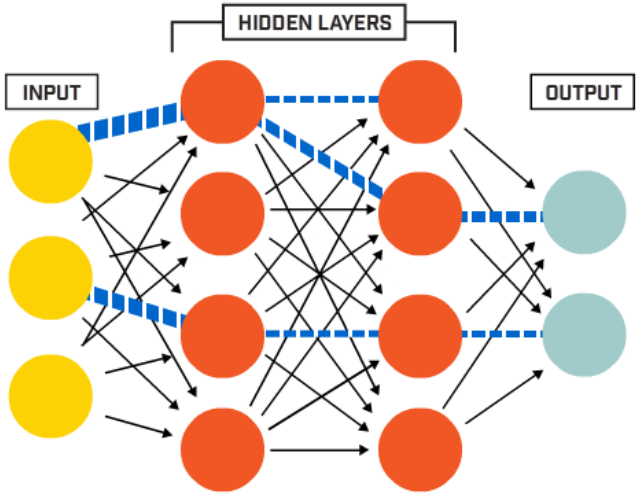
RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

DEEP LEARNING



Dr S Anupama Kumar
Associate Professor
Dept of AIML
RVCE

Deep Learning

Perceptron

- The Perceptron is the simplest type of artificial neuron, introduced by Frank Rosenblatt(1958).
- It is a binary classifier that decides whether an input belongs to one class or another based on a linear combination of inputs and weights.
- Perceptron consists of a single layer of input nodes that are fully connected to a layer of output nodes.
- This foundational model has played a crucial role in the development of more advanced neural networks and machine learning algorithms.

Deep Learning

Types of Perceptron

Single-Layer Perceptron is a type of perceptron is limited to learning linearly separable patterns.

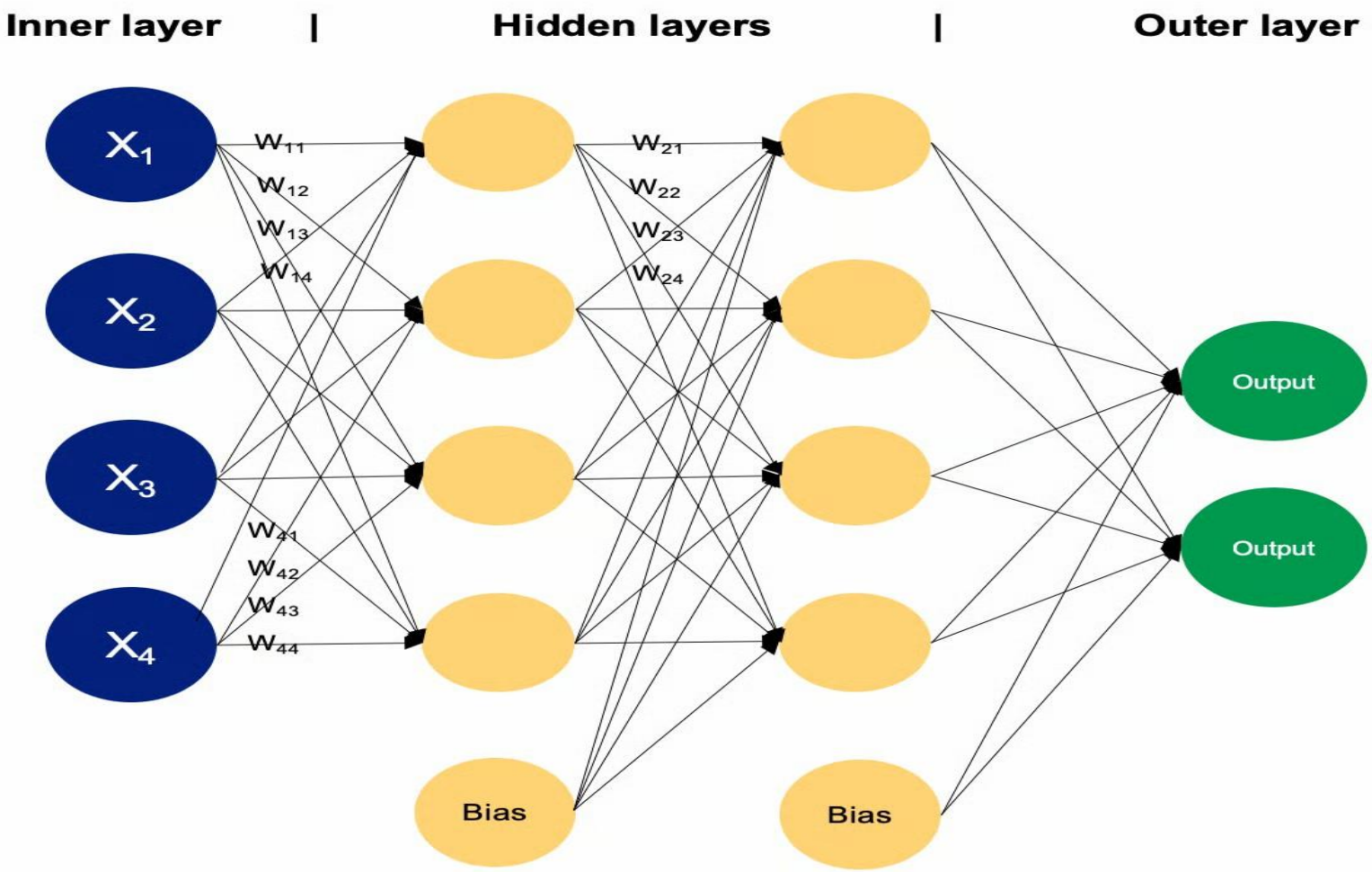
- It is effective for tasks where the data can be divided into distinct categories through a straight line.
- While powerful in its simplicity, it struggles with more complex problems where the relationship between inputs and outputs is non-linear.

Multi-Layer Perceptron possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

- MLP is a feedforward neural network consisting of fully connected neurons with a nonlinear kind of activation function.
- It is widely used to distinguish data that is not linearly separable.

Deep Learning

Multilayer Perceptron



Deep Learning

Multi Layer Perceptron

Input layer

- The input layer consists of nodes or neurons that receive the initial input data.
- Each neuron represents a feature or dimension of the input data.
- Each neuron in the input layer represents one feature.
- The number of neurons in the input layer is determined by the dimensionality of the input data.
- Neurons in the input layer do not perform any computations; they simply pass the input values to the neurons in the first hidden layer.

Deep Learning

Multi Layer Perceptron

Hidden layer

- The hidden layers of an MLP consist of interconnected neurons that perform computations on the input data.
- Each neuron in a hidden layer receives input from all neurons in the previous layer.
- The inputs are multiplied by corresponding weights, denoted as w. The weights determine how much influence the input from one neuron has on the output of another.
- In addition to weights, each neuron in the hidden layer has an associated bias, denoted as b.
- The bias provides an additional input to the neuron, allowing it to adjust its output threshold. Like weights, biases are learned during training.

Deep Learning

Multi Layer Perceptron

- For each neuron in a hidden layer or the output layer, the weighted sum of its inputs is computed.

$$\textbf{Weighted Sum} = \sum_{i=1}^{n} (w_i * x_i) + b$$

The weighted sum is then passed through an activation function, denoted as f.

Activation function

- Each neuron in the hidden layers and the output layer applies an activation function to its weighted sum of inputs.
- The activation function introduces nonlinearity into the network, allowing it to learn and represent complex relationships in the data.
- The activation function determines the output range of the neuron and its behavior in response to different input values.
- The choice of activation function depends on the nature of the task and the desired properties of the network.

Deep Learning

Multi Layer Perceptron

Output layer

- The output layer of an MLP produces the final predictions or outputs of the network.
- The number of neurons in the output layer depends on the task being performed (e.g., binary classification, multi-class classification, regression).
- Each neuron in the output layer receives input from the neurons in the last hidden layer and applies an activation function.
- This activation function is usually different from those used in the hidden layers and produces the final output value or prediction.

Adjustments - Stochastic gradient descent (SGD)

- During the training process, the network learns to adjust the weights associated with each neuron's inputs to minimize the discrepancy between the predicted outputs and the true target values in the training data.
- By adjusting the weights and learning the appropriate activation functions, the network learns to approximate complex patterns and relationships in the data, enabling it to make accurate predictions on new, unseen samples.

Deep Learning

Multi Layer Perceptron – Back Propagation

- Backpropagation is short for “backward propagation of errors.”
- SGD involves updating the network's parameters iteratively based on the gradients computed during each batch of training data.
- Instead of computing the gradients using the entire training dataset, SGD computes the gradients using small random subsets of the data called mini-batches.

Deep Learning

Multi Layer Perceptron – Back Propagation

Forward pass:- input data is fed into the neural network, and the network's output is computed layer by layer. Each neuron computes a weighted sum of its inputs, applies an activation function to the result, and passes the output to the neurons in the next layer.

Loss computation: - the network's output is compared to the true target values, and a loss function is computed to measure the discrepancy between the predicted output and the actual output.

Backward pass (gradient calculation):- the gradients of the loss function with respect to the network's parameters (weights and biases) are computed using the chain rule of calculus.

Parameter update: Once the gradients have been computed, the network's parameters are updated in the opposite direction of the gradients in order to minimize the loss function. This update is typically performed using an optimization algorithm such as stochastic gradient descent (SGD), that we discussed earlier.

Iterative process: Steps 1-4 are repeated iteratively for a fixed number of epochs or until convergence criteria are met.

During each iteration, the network's parameters are adjusted based on the gradients computed in the backward pass, gradually reducing the loss and improving the model's performance.